

Tushar Jindal

236-412-0273 | tushar.bzp05@gmail.com | linkedin.com/in/tushar-jindal-97602420b/ | github.com/tjindl

EDUCATION

University of British Columbia

Bachelor of Science in Data Science

Vancouver, BC

Graduating Summer 2028

EXPERIENCE

Web Developer

June 2026 – Present

Science Undergraduate Society (SUS) at UBC

Vancouver, BC (Hybrid)

- Use React, HTML and CSS to administer and maintain SUS website and digital signage network
- Work with UI/UX designers and other developers on weekly working-group meetings to improve page layout, functionality, user experience, support annual hackathon portals, bots etc.

Fullstack Developer Intern

December 2025 – March 2026

The F Word AI*

Fremont, CA (Remote)

- Designed and shipped a suite of four production Telegram bots from the ground up — user onboarding, an FAQ assistant, daily personalized job alerts, and a broadcast notification service for the main channel — covering the full user lifecycle from signup to ongoing engagement on a live fashion-tech platform.
- Using Playwright, created a custom web scraper that processed about 100,000 pages per month and reduced infrastructure costs by about 80% when compared to managed scraping services.
- Built the supporting full-stack infrastructure (FastAPI backend, Dockerized deployment, and CI/CD via GitHub Actions) and integrated the OpenAI API for batch processing, collaborating with a team of developers.

PROJECTS

BitSmith | *Python, C++, PyTorch, HuggingFace transformers, Quantization(AWQ), Model Compression, ARM NEON SIMD*

- Used AWQ quantization on GPT-2, end-to-end INT4 LLM compression pipeline design, resulting in a 50% reduction in model size (248.9 MB → 124.1 MB) with a perplexity degradation of only +0.3% compared to the FP16 baseline.
- Using ARM NEON SIMD INT4 kernels and PyBind11, made a zero-dependency C++ inference engine. Processes 16 INT4 values per NEON iteration across four parallel accumulators to provide real-hardware throughput at 5.8 tok/s on Apple Silicon.
- Designed a graph-theory-guided structured pruning method that uses PageRank bottleneck protection and per-layer importance scoring to physically remove up to 44/144 attention heads and 18K/36K MLP neurons for actual FLOP reduction, not just weight zeroing.

ASL Detector | *Python, TensorFlow, CNN, React.js, Flask, OpenCV, MediaPipe, Data Preprocessing, REST API*

- Trained and designed CNN on Sign MNIST Dataset using 3 convolutional blocks with BatchNormalization, MaxPooling, and Dropout regularization to achieve 99.9% accuracy across 24 ASL letter classes.
- Created a full-stack, real-time ASL recognition web application that supports both live gesture recognition and image upload using a Flask REST API backend and a React frontend.
- Used TensorFlow/Keras to implement an end-to-end deep learning pipeline that included grayscale preprocessing, pixel normalization, and Softmax classification across 24 output classes.

Handly | *Python, scikit-learn, MediaPipe, OpenCV, Random Forest, AppleScript, PyTorch, Feature Engineering.*

- Trained a Random Forest classifier using 63-feature MediaPipe hand landmark vectors, outperforming a concurrently trained MLP baseline and achieving 99.3% test accuracy across 9 gesture classes.
- Created a comprehensive real-time gesture recognition pipeline from custom data collection (live HUD + per-class progress tracking) to inference, using AppleScript to map six different gestures to macOS system controls.
- 8-frame debouncing and 1.2s cooldown logic were used to engineer production-grade stability, removing unintentional triggers during continuous webcam inference.

TECHNICAL SKILLS

Languages: Java, Python, C/C++, SQL (Postgres), JavaScript, TypeScript, HTML/CSS, R

ML/AI: PyTorch, TensorFlow, scikit-learn, OpenCV, MediaPipe, HuggingFace Transformers

Frameworks: React, Node.js, Flask, JUnit, FastAPI

Developer Tools: Git, Docker, Azure, VS Code, PyCharm, IntelliJ

Libraries: Pandas, NumPy, Matplotlib, spaCy, PyBind11